

Class 19: Lab – Streaming Pasteurization Sensors

Data-driven Systems Engineering

Agenda

- Lab goal: go from synthetic sensor generation to a live stream you can consume.
- Walk through 'synth_sensors.py': states, sensors, and the derived 'dTdt' feature.
- Walk through 'serving.py': batch, point, and streaming endpoints.
- Guided exercises: run, inspect, request, and stream.
- What to observe, and common pitfalls.

Learning goals

- Run the pasteurization synthetic sensor generator and inspect its output.
- Explain how a Flask endpoint turns a batch simulation into a live stream.
- Connect the state machine from Class 13 to actual code that implements it.
- Practice validating a data-generating service before trusting anything built on top of it.

Course Map and Running Cases



Today in the course

Focus: Hands-on practice: generating and serving the streaming sensor case end to end.

Bridge: This is a lab session: it takes the data-type and online-learning ideas from Classes 13 and 18 and turns them into a running service you operate yourself.

Fraud case

The fraud case has no streaming lab counterpart, but the habits practiced here, validating a generator, testing an endpoint, still apply directly to any API-based service.

Pasteurization case

The pasteurization case is the entire subject of this lab: 'synth_sensors.py' generates it, 'serving.py' streams it.

Course thread: Everything from data types to online learning becomes concrete only once you can run the pipeline that produces the data.

Lab overview and setup

1. From the repository root, install dependencies: `'pip install -r requirements.txt'`.
2. Two files matter today: `'pasteurization/synth_sensors.py'` and `'pasteurization/serving.py'`.
3. `'synth_sensors.py'` has no Flask dependency; it can be run and inspected standalone.
4. `'serving.py'` imports functions directly from `'synth_sensors.py'` and exposes them over HTTP.

The sensors and states, in code

```
# Sensors (7): Temperature, pH, Kappa, Mu, Tau, Q_in, Q_out, P
# Derived: dT/dt

PROD_RANGES = {
    "Idle":      (60, 120),
    "Fill":      (30, 60),
    "HeatUp":    (30, 45),
    "Hold":      (15, 25),
    "Cool":      (30, 60),
    "Discharge": (60, 120),
}
```

- From 'pasteurization/synth_sensors.py': seven physical sensors, six production states, each with a randomized duration range in seconds.

The batch lifecycle as a state machine



- Every function named 'seg_*' in 'synth_sensors.py' implements exactly one of these six boxes.

Exercise 1: generate and inspect a batch

1. Run `'python synth_sensors.py'` from the `'pasteurization/'` directory.
2. Open the resulting `'synthetic_pasteurization_signals.csv'` and check the `'state'` column against the six states above.
3. Plot `'T'` and `'dTdt'` for a single `'batch_id'` and identify the `'HeatUp'` and `'Cool'` phases visually.
4. Confirm that `'dTdt'` changes sign exactly where the temperature trend reverses.

The derived feature, in code

```
def simulate_batch(bid, start_time=0.0):  
    ...  
    frames = simulate_production_cycle(pdur, ctx)  
    df, t = to_df(frames, start_time, bid)  
    df["dTdt"] = df["T"].diff().fillna(0.0) / DT  
    return df, t
```

- This is the exact construction from Class 15's “feature construction for streams” section, now something you run yourself.

Exercise 2: serve a batch over HTTP

1. Start the Flask service: `'python serving.py'` (listens on port 8001).
2. Request a full simulated batch: `'GET /batch'`.
3. Request a single simulated reading at a chosen time: `'GET /point?t=45'`.
4. Vary `'t'` across the six state durations and confirm the returned `'state'` field changes accordingly.

Serving a single point, in code

```
@app.route("/point")
def point():
    try:
        t = float(request.args.get("t", 0))
    except ValueError:
        return jsonify({"error": "Invalid t parameter"}), 400

    data = simulate_point(t)
    return jsonify(data)
```

- From 'pasteurization/serving.py': input validation ('try'/'except') is already present here, a good pattern to notice and reuse.

Exercise 3: consume the live stream

1. With 'serving.py' still running, request 'GET /stream' with 'curl -N http://localhost:8001/stream'.
2. Observe one JSON event arriving per second, in the Server-Sent Events format.
3. Alternatively, launch 'streamlit run streamlit/Main.py' and open the Pasteurization dashboard page, which consumes this same '/stream' endpoint.
4. Compare the raw 'curl' output to what the dashboard renders.

The streaming endpoint, in code

```
@app.route("/stream")
def stream():
    df, _ = simulate_batch(BATCH_ID)

    def generate():
        for _, row in df.iterrows():
            yield f"data: {json.dumps(row.to_dict())}\n\n"
            time.sleep(STREAM_DT)

    return Response(generate(), mimetype="text/event-stream")
```

- From 'pasteurization/serving.py': a whole batch is simulated first, then replayed one row per second as a live event stream.

What to observe while streaming

- State transitions should follow the fixed order: Idle, Fill, HeatUp, Hold, Cool, Discharge.
- 'dTdt' should be near zero during 'Idle' and 'Hold', and clearly non-zero during 'HeatUp' and 'Cool'.
- 'Q_in' is only non-zero during 'Fill'; 'Q_out' is only non-zero during 'Discharge'.
- Every sensor stays within the fixed y-axis ranges used by the Streamlit dashboard.

Common pitfalls in this lab

- Forgetting `'threaded=True'` when running Flask can make the `'/stream'` endpoint block other requests.
- Port conflicts: `'serving.py'` defaults to 8001; check nothing else is already bound to it.
- The Streamlit dashboard's sidebar IP and port must match wherever `'serving.py'` is actually running.
- `'/stream'` regenerates a new batch every time it is called; two simultaneous clients will not see identical data.

Repo activity

- 'pasteurization/synth_sensors.py' and 'pasteurization/serving.py' are the two files this entire lab is built from.
- 'streamlit/pages/App_Pasteurization.py' is the ready-made client that consumes '/stream' for live plotting.

Question

What would you change in 'synth_sensors.py' to simulate a sensor fault or a slow drift, so that Class 18's ADWIN detector would have something real to catch?

- There is no single correct answer; the goal is to connect a code change to a detectable statistical effect.

Papers and materials

Flask documentation; Flask tutorial; Streamlit documentation; pandas documentation.

From This Class to the Next

What stays with us

- A synthetic generator plus a thin Flask layer is enough to turn a state-machine model into a live, streamable service.
- Input validation and clear route boundaries are software engineering habits, not extras, even in a small simulation service.
- Running and probing a service yourself surfaces assumptions that reading the code alone does not.

Project use

Project teams building a streaming or simulation component can reuse this generate-then-serve pattern directly.

Next connection

Class 20 asks how a trained model, not just a data generator, gets served the same way, and bridges into the MLOps block that follows.

Course thread: Whether the payload is a sensor reading or a model prediction, the serving concerns, validation, format, and reliability, are the same.